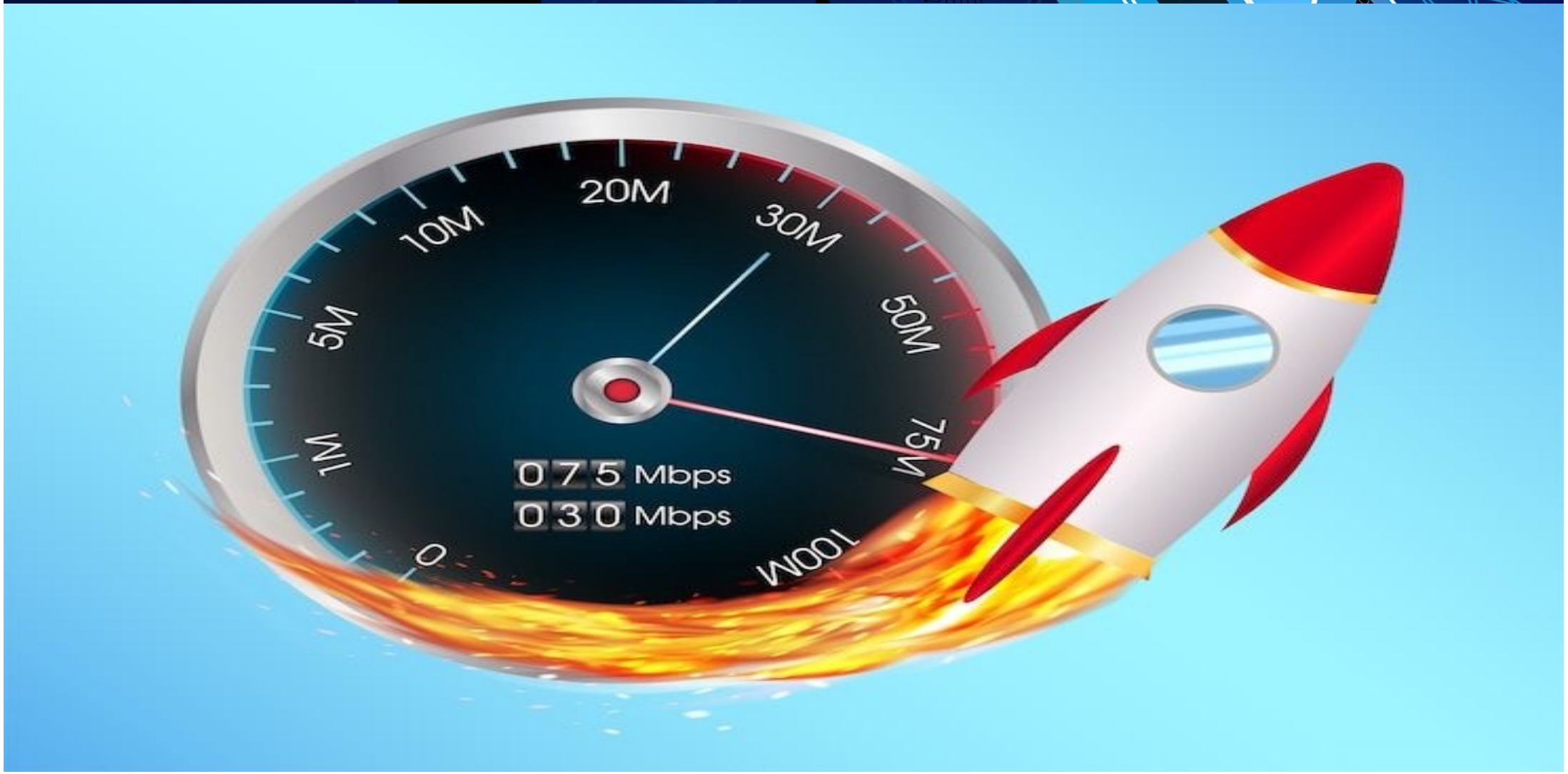
# Le partitionnement



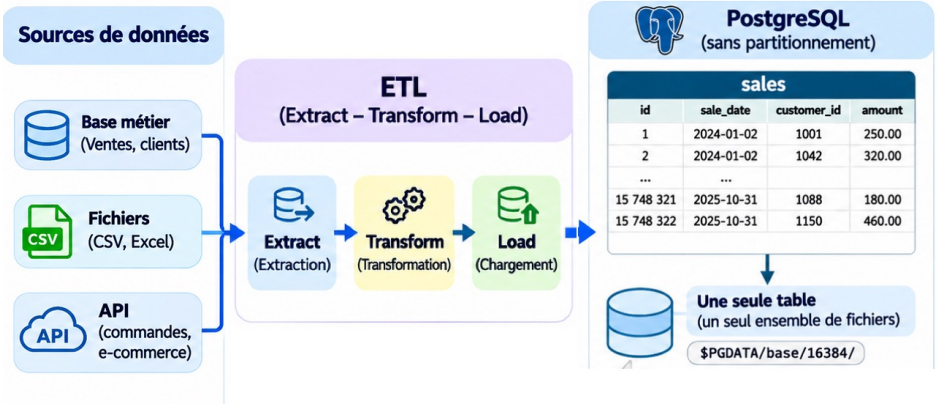
# Pourquoi partitionner les données ?

Une entreprise de e-commerce souhaite analyser ses ventes afin de suivre son activité et d'améliorer ses décisions commerciales.

Les données proviennent de plusieurs sources (bases métier, fichiers CSV, API) et sont centralisées dans une base PostgreSQL à l'aide d'un processus **ETL (Extract, Transform, Load)**.

**Besoins métier**

- Analyser le chiffre d'affaires par jour, mois et année.
- Identifier les tendances et les périodes de forte activité.
- Comparer les performances commerciales entre différentes périodes.
- Conserver l'historique des ventes pour les analyses.




**Données et volumétrie**

- **Table sales** : historique des transactions commerciales.
- **120 millions de ventes par an**, soit environ 10 millions par mois.
- Plusieurs années d'historique à conserver.

# Pourquoi partitionner les données ?

- **Quels problèmes peut-on rencontrer avec une table contenant plusieurs milliards de lignes ?**
  - Volumétrie importante.
  - Requêtes potentiellement coûteuses.
  - Maintenance des index.
  - Suppression des anciennes données.
  - Opérations de maintenance longues.
  - Capacité de stockage.

Une table volumineuse n'est pas nécessairement lente : les performances dépendent notamment des index, des requêtes et des ressources disponibles.

# le partitionnement

Le partitionnement consiste à diviser une grande table logique en plusieurs tables physiques appelées **partitions**. Du point de vue de l'application, les données restent accessibles à travers une seule table (la table parent).




Chaque partition possède son propre nom et peut éventuellement utiliser des options de stockage différentes.

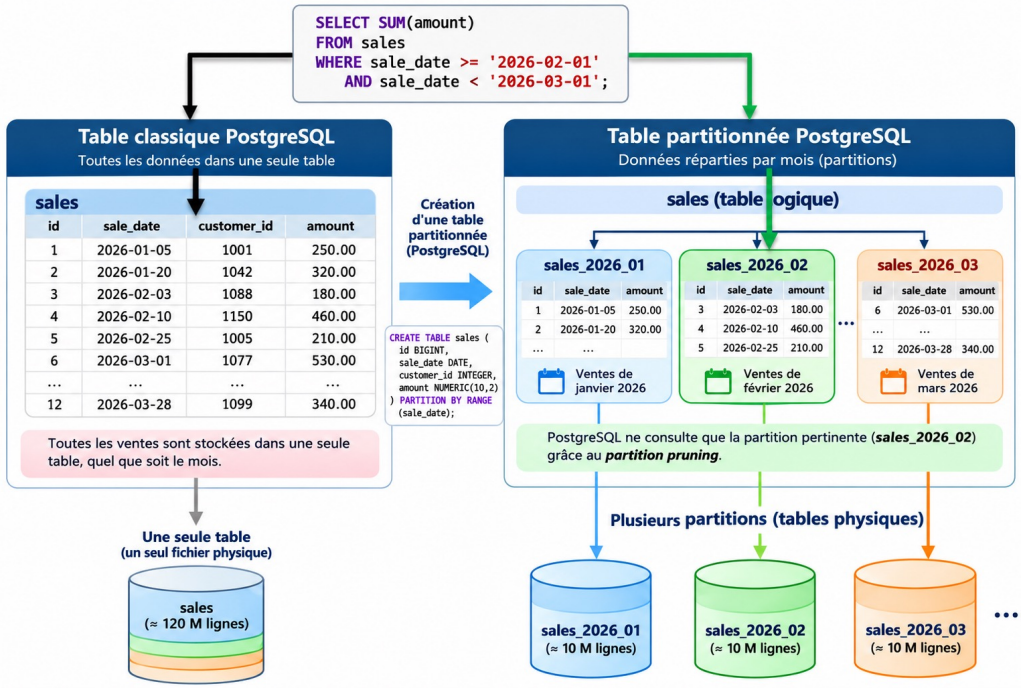
Le partitionnement offre plusieurs avantages :

- **Amélioration des performances**, aussi bien en lecture qu'en écriture, grâce au traitement sur des volumes plus petits.
- **Gestion facilitée du cycle de vie des données**, en permettant d'archiver, déplacer ou supprimer des partitions entières très rapidement.

On utilise généralement le partitionnement lorsque :

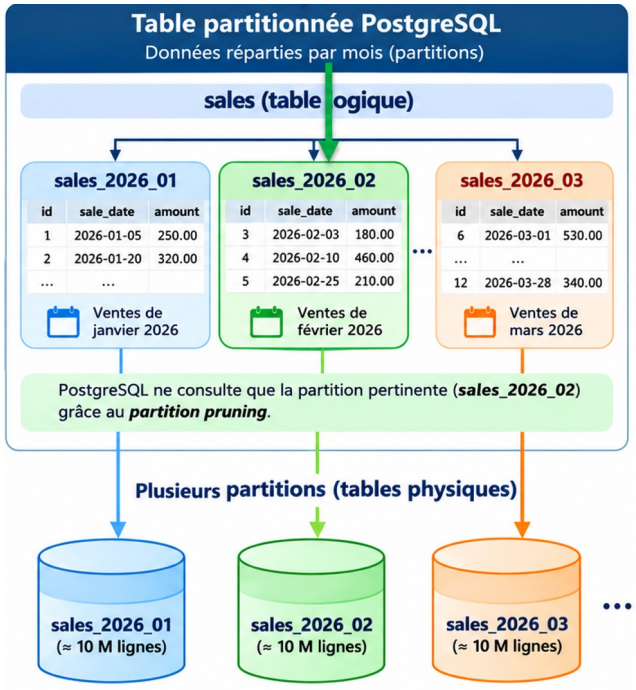
- une table dépasse **plusieurs gigaoctets** (environ 2 Go ou plus),
- la table contient des **données d'historique ou d'archivage**,
- il est nécessaire de **supprimer régulièrement de gros volumes de données**.

# Le partitionnement




- Le SGBD peut éliminer les partitions inutiles grâce au *partition pruning*.
- Au lieu de parcourir les N partitions, il ne consulte que celle de février.
- Attention : cela ne signifie pas que le SGBD ne lit que les lignes pertinentes dans cette partition. Les index et le plan d'exécution restent importants.

# Le partitionnement



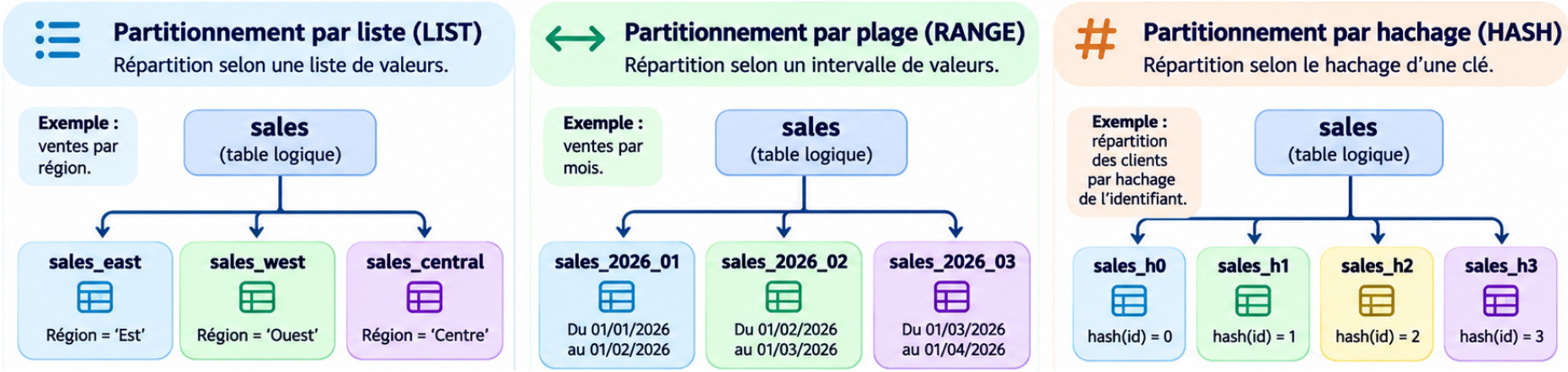

**Comment les données sont-elles réparties dans les partitions ?**

- Les données sont partitionnées à partir d'une **clé de partitionnement**, c'est-à-dire une colonne (ou un petit ensemble de colonnes) dont la valeur détermine dans quelle partition chaque ligne sera placée.
- Cette clé correspond très souvent une superclé ( un ensemble de colonnes incluant la **clé primaire)** de la table.

- ⚠ **Point important :**
- Pour que le SGBD puisse exploiter efficacement le partitionnement (pruning), les requêtes en lecture ou en écriture doivent inclure **la clé de partitionnement** dans la clause **WHERE**.
- Sans cette clé, le moteur risque de devoir parcourir *toutes* les partitions, réduisant l'intérêt de l'optimisation.

# Le partitionnement

## Les différents algorithmes de partitionnement




**1. Partitionnement par RANGE (plages de valeurs)**

Découpe la table selon des intervalles ordonnés.
→ *Très utilisé pour les données temporelles.*

**Exemples :** par date (mois, année), par identifiant croissant

**2. Partitionnement par LISTE**

Répartit les lignes selon une liste explicite de valeurs.
→ *Pratique lorsque les groupes sont discrets et connus à l'avance.*
**Exemples :** pays = {FR, ES, IT} type_client = {pro, particulier}

**3. Partitionnement par HASH**

Utilise une fonction de hachage sur une colonne pour répartir uniformément les données.
*Utile pour éviter les partitions déséquilibrées.*

# Comment choisir sa stratégie de partitionnement ?

Le choix de la stratégie de partitionnement est déterminant pour les performances et la gestion des données.

1. Analyser les requêtes de l'application

- Identifier les colonnes fréquemment utilisées dans les filtres (WHERE).
- Favoriser le partition pruning : éliminer les partitions inutiles lors de l'exécution des requêtes.
- Limiter les requêtes nécessitant de parcourir toutes les partitions.

2. Choisir une clé de partitionnement adaptée

- Privilégier une clé cohérente avec les requêtes les plus fréquentes.
- Tenir compte de la volumétrie et de la répartition des données.
- Éviter les déséquilibres importants entre partitions lorsqu'ils pénalisent les performances.

# Comment choisir sa stratégie de partitionnement

3. Choisir la méthode de partitionnement

- RANGE : adapté aux recherches par intervalles (dates, périodes, valeurs numériques).
- HASH : favorise une distribution homogène des données entre les partitions.
- LIST : permet de regrouper explicitement les données selon des valeurs prédéfinies (régions, catégories, pays).

À retenir : une bonne stratégie de partitionnement dépend avant tout des requêtes, de la volumétrie et du cycle de vie des données.

# Les bénéfices du partitionnement




**Maintenance**

- Chaque partition possède ses propres structures physiques (données et index).
- Les opérations de maintenance peuvent être réalisées indépendamment sur chaque partition : VACUUM, ANALYZE, REINDEX.

**Gestion du cycle de vie**

- **Détacher** une partition : elle devient une table indépendante, sans être accessible via la table parent.
- **Archiver** une partition : possibilité de déplacer ses données vers un stockage moins coûteux.
- **Réattacher** une partition si nécessaire.
- **Supprimer** rapidement les données anciennes en supprimant une partition.

Performance

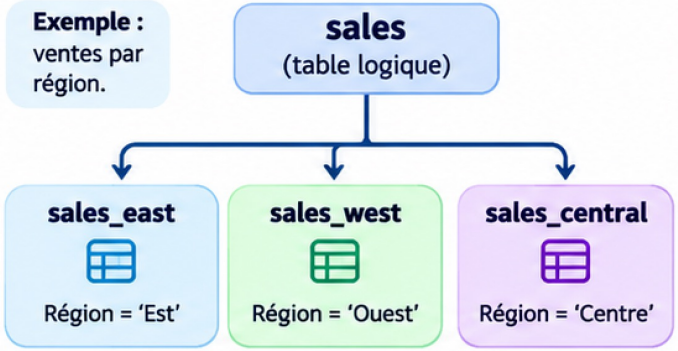
- **Partition pruning** : seules les partitions pertinentes sont consultées.
- **Index locaux** : chaque partition possède ses propres index physiques, généralement plus petits.
- **Maintenance des index** : possibilité de reconstruire les index d'une partition indépendamment.
- ⚠ **Contraintes d'unicité**
  - **PostgreSQL** : pas d'index global physique. Une contrainte PRIMARY KEY ou UNIQUE sur la table partitionnée doit inclure toutes les colonnes de partitionnement.
  - **Oracle** : prise en charge des index locaux et globaux, permettant notamment de garantir une unicité indépendante de la clé de partitionnement avec un index unique global.

# Implémentation d’une table partitionnée

## Partitionnement par liste (LIST)

Répartition selon une liste de valeurs.

**Exemple :** ventes par région.




**Exemple SQL (PostgreSQL)**

```
CREATE TABLE  sales (
  id BIGINT, region TEXT, amount NUMERIC
) PARTITION BY LIST (region);

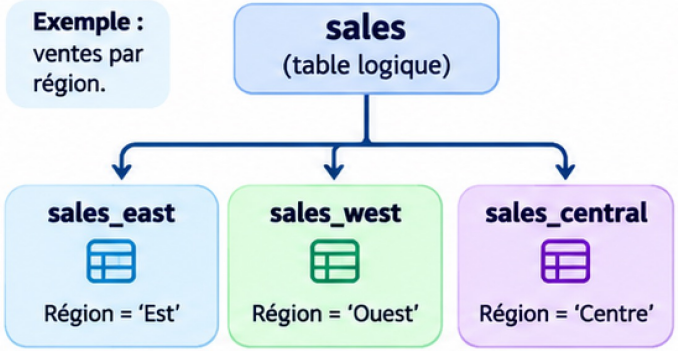
CREATE TABLE sales_east    PARTITION OF sales
  FOR VALUES IN ('Est');
```

# Implémentation d’une table partitionnée

## Partitionnement par liste (LIST)

Répartition selon une liste de valeurs.

**Exemple :** ventes par région.




**Exemple SQL (PostgreSQL)**

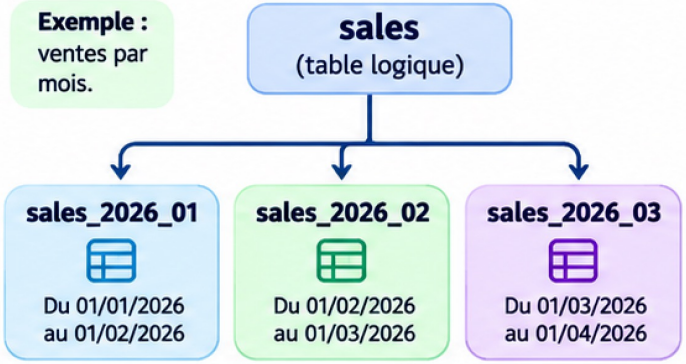
```
CREATE TABLE  sales (
  id BIGINT, region TEXT, amount NUMERIC
) PARTITION BY LIST (region);

CREATE TABLE sales_east    PARTITION OF sales
  FOR VALUES IN ('Est');
```

## Partitionnement par plage (RANGE)

Répartition selon un intervalle de valeurs.

**Exemple :** ventes par mois.




**Exemple SQL (PostgreSQL)**

```
CREATE TABLE sales (
  id BIGINT, sale_date DATE, amount NUMERIC
) PARTITION BY RANGE (sale_date);

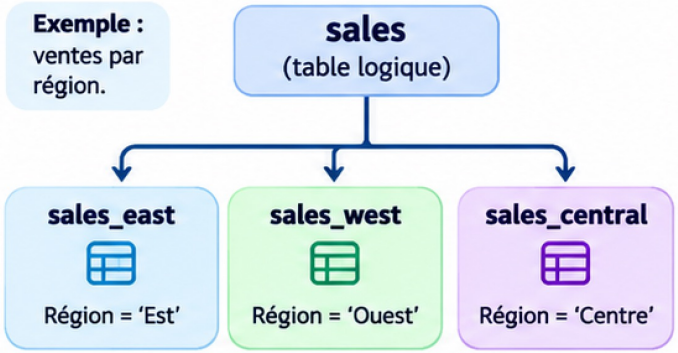
CREATE TABLE sales_2026_01 PARTITION OF sales
   FOR VALUES FROM ('2026-01-01')
     TO ('2026-02-01');
```

# Implémentation d'une table partitionnée

## Partitionnement par liste (LIST)

Répartition selon une liste de valeurs.

**Exemple :** ventes par région.




**Exemple SQL (PostgreSQL)**

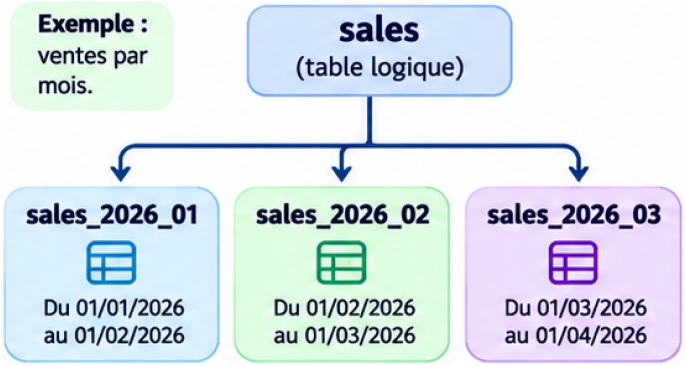
```
CREATE TABLE  sales (
  id BIGINT, region TEXT, amount NUMERIC
) PARTITION BY LIST (region);

CREATE TABLE sales_east    PARTITION OF sales
  FOR VALUES IN ('Est');
```

## Partitionnement par plage (RANGE)

Répartition selon un intervalle de valeurs.

**Exemple :** ventes par mois.




**Exemple SQL (PostgreSQL)**

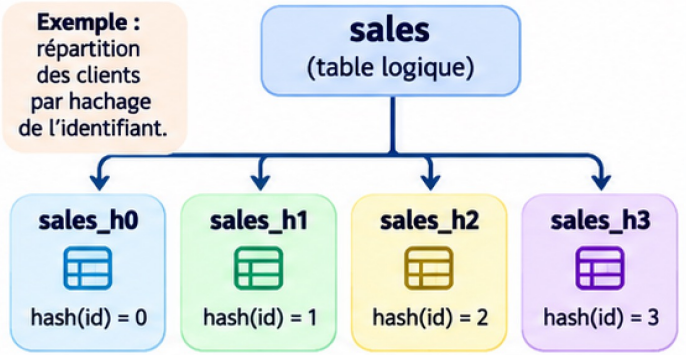
```
CREATE TABLE sales (
  id BIGINT, sale_date DATE, amount NUMERIC
) PARTITION BY RANGE (sale_date);

CREATE TABLE sales_2026_01 PARTITION OF sales
   FOR VALUES FROM ('2026-01-01')
     TO ('2026-02-01');
```

## Partitionnement par hachage (HASH)

Répartition selon le hachage d'une clé.

**Exemple :** répartition des clients par hachage de l'identifiant.




**Exemple SQL (PostgreSQL)**

```
CREATE TABLE sales (
 id BIGINT, customer_id BIGINT, amount NUMERIC
) PARTITION BY HASH (customer_id);

CREATE TABLE sales_h0  PARTITION OF sales
  FOR VALUES WITH (MODULUS 4, REMAINDER 0);
```

# Evolution du partitionnement



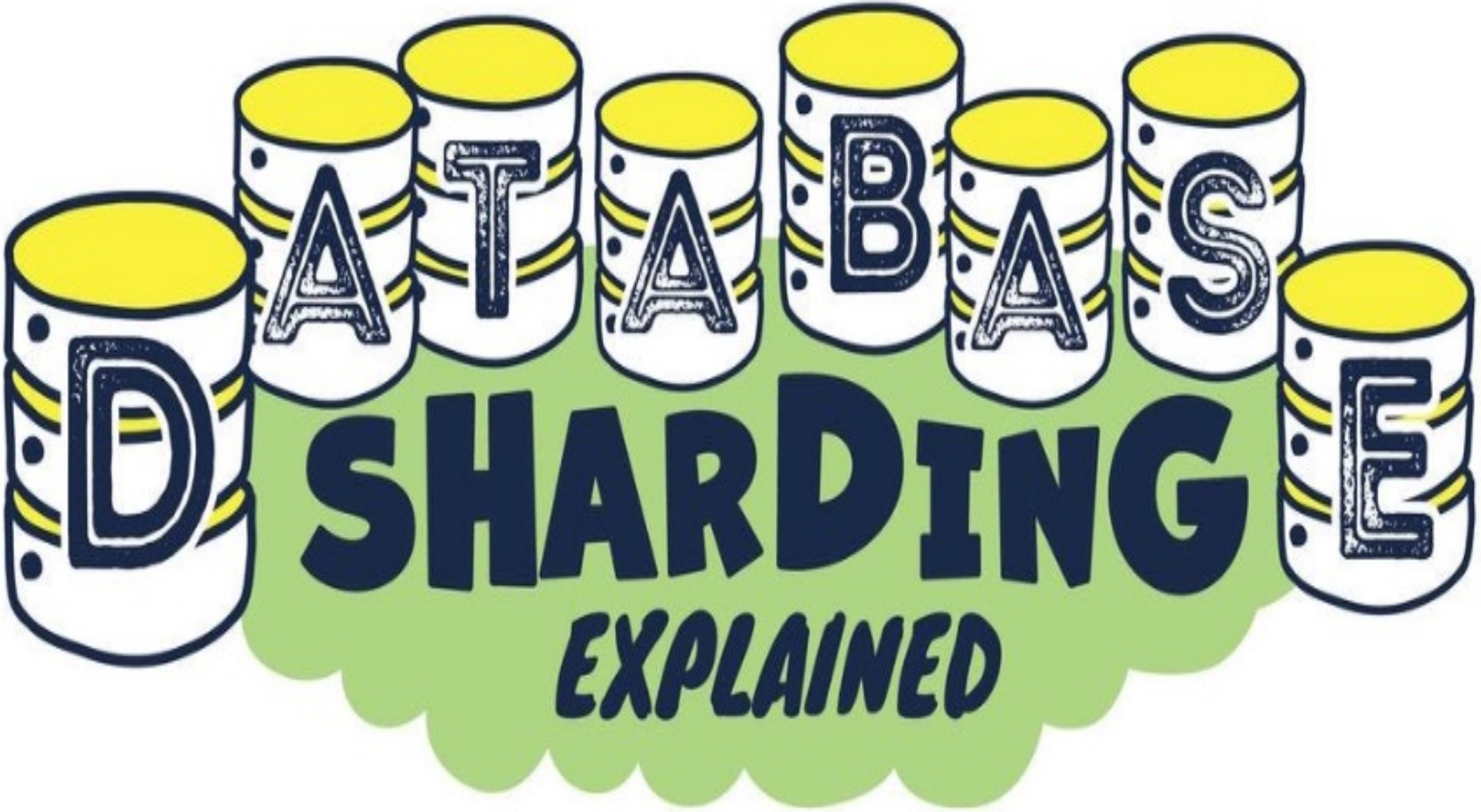

Malgré notre partitionnement, nous rencontrons de nouvelles difficultés :

- Les insertions saturent les ressources du serveur.
- Les requêtes analytiques consomment trop de CPU.
- Les opérations d'écriture deviennent un goulot d'étranglement.

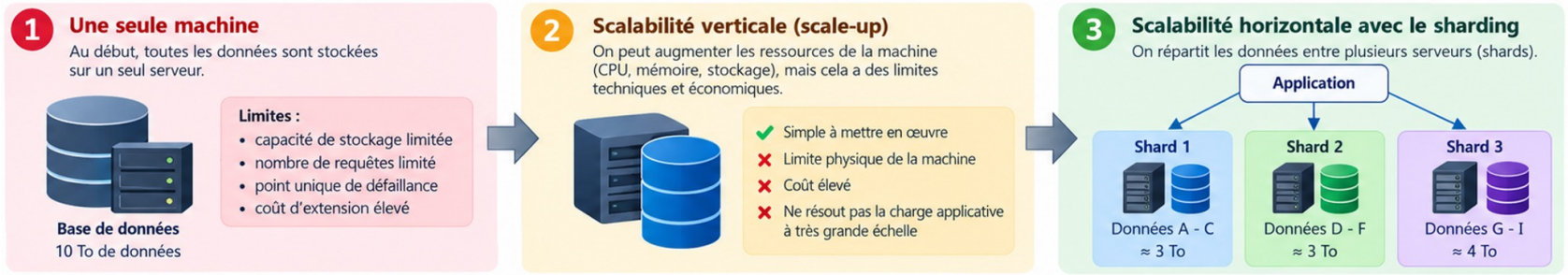
Que pouvons-nous faire si ce serveur ne suffit plus ?

# TP01 - Partitionnement




DATABASE
SHARDING
EXPLAINED

# Le sharding



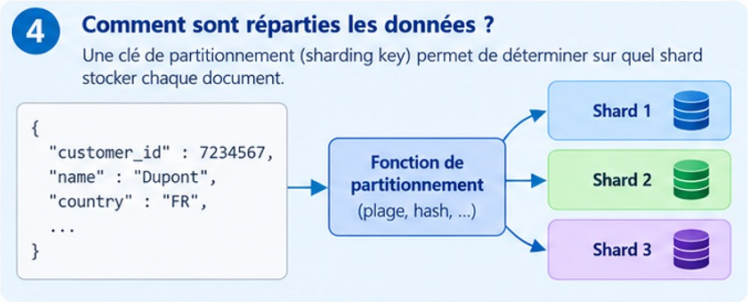

Le sharding est une technique de partitionnement horizontal qui consiste à répartir un ensemble de données entre plusieurs serveurs/machines, appelés shards.
Contrairement au partitionnement précédent, le sharding réparti les données sur plusieurs instances.

Chaque shard ne contient donc qu'une partie des données.

Cela permet potentiellement de distribuer :
- Le stockage.
- Les écritures.
- Les lectures.
- Les traitements.

Mais cette distribution ajoute de la complexité.

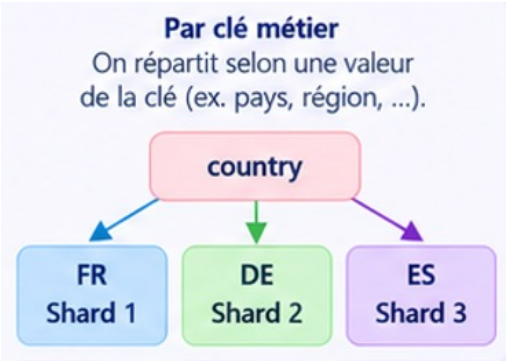
# Le sharding




Le sharding ne consiste pas seulement à découper les données. Il faut également disposer d'une stratégie permettant de déterminer où chaque donnée doit être stockée et retrouvée.

Cette stratégie est basé sur la clé de partitionnement , La clé de partitionnement (*partition key* ou *shard key*) est une donnée utilisée par le système pour déterminer sur quel shard une donnée doit être stockée et recherchée.

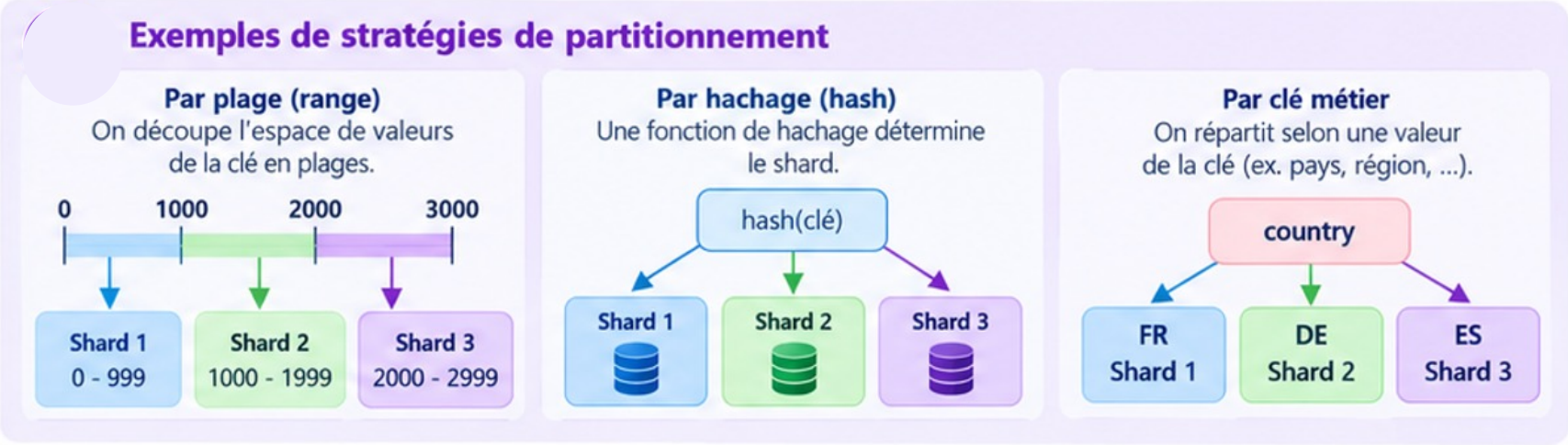
Par exemple :



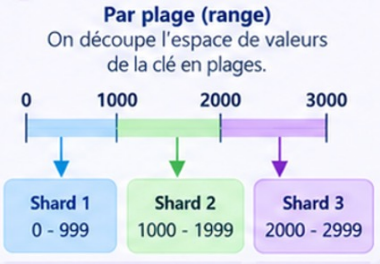
# Le sharding

La fonction de partitionnement est la règle ou l'algorithme qui utilise la clé de partitionnement pour déterminer sur quel shard une donnée doit être placée.

La clé indique sur quelle information répartir les données ; la fonction de partitionnement indique comment utiliser cette information pour choisir le shard.

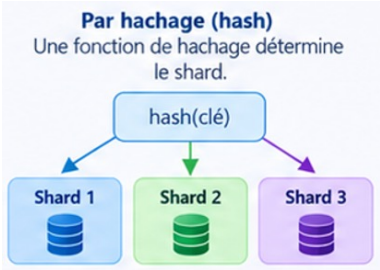


# Partitionnement horizontal



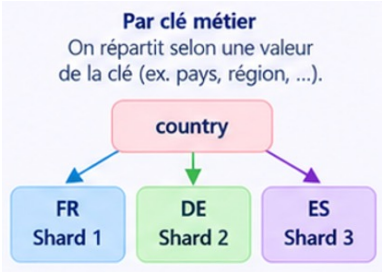

Le partitionnement par intervalle consiste à diviser les valeurs de la clé de partitionnement en plusieurs plages ordonnées.

Chaque plage est ensuite associée à une partition ou à un shard.




Le partitionnement par hachage consiste à appliquer une fonction de hachage à la clé de partitionnement afin de déterminer où stocker la donnée.

Le partitionnement par hachage privilégie une distribution homogène




Le partitionnement par liste consiste à associer explicitement certaines valeurs de la clé de partitionnement à une partition.

Contrairement au partitionnement par intervalle ou par hachage, la répartition repose ici sur une liste de valeurs prédéfinies.

# Partitionnement horizontal



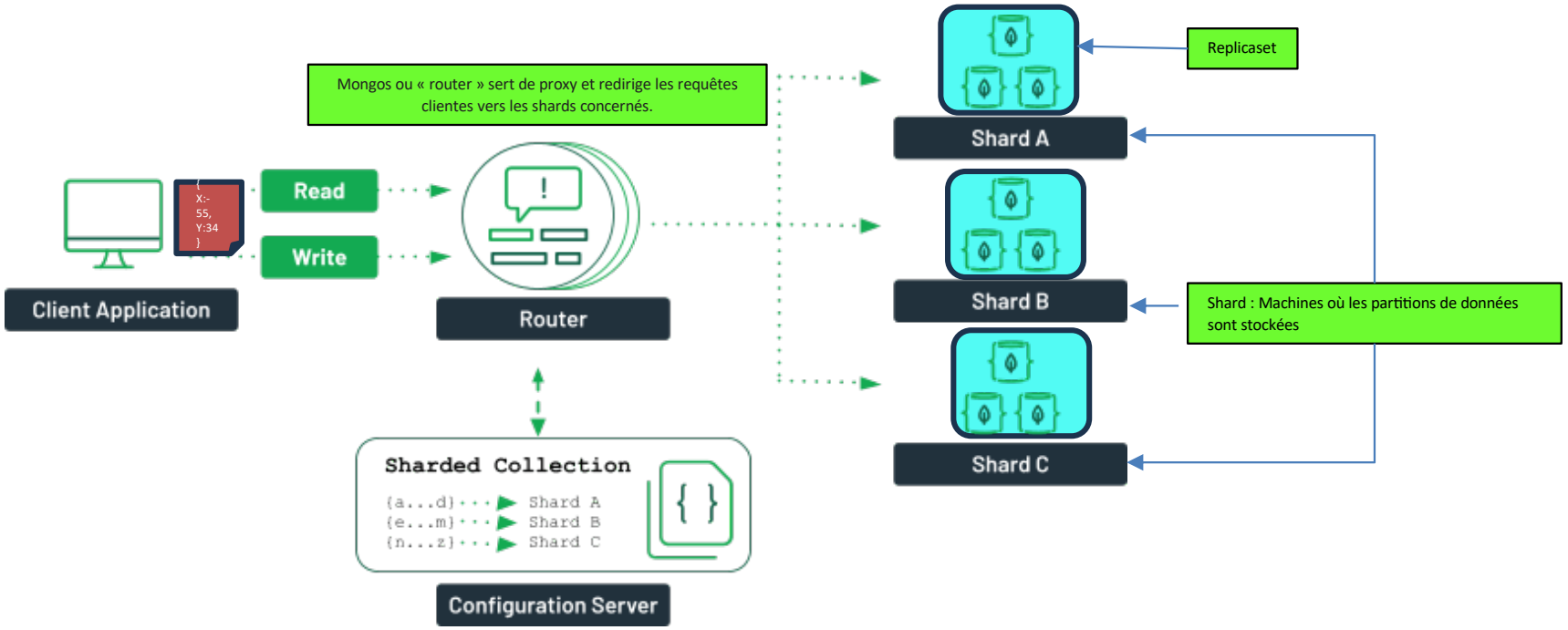

- Prévoir l'évolution et le rééquilibrage
  Le volume et la charge évoluent. Il faut pouvoir ajouter des nœuds et redistribuer les partitions sans interruption majeure.

- Ne pas confondre sharding et haute disponibilité
  Le sharding distribue les données et la charge. La réplication fournit la redondance et la tolérance aux pannes. Les deux mécanismes sont généralement complémentaires.

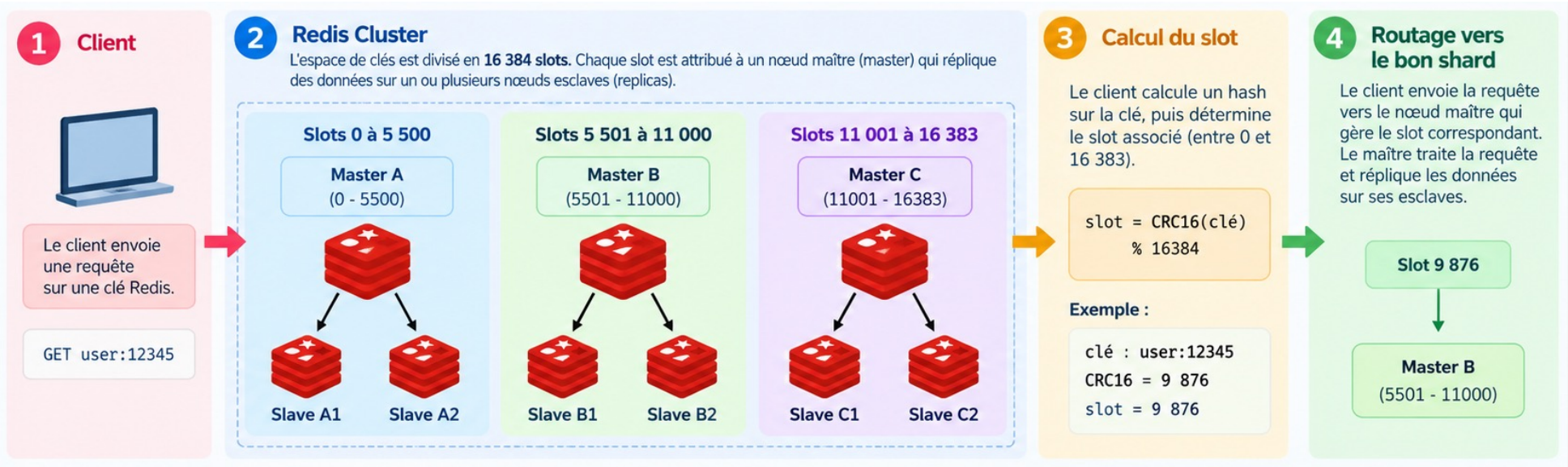
- Éviter les déséquilibres et les hotspots
  Un shard ne doit pas concentrer la majorité des données, des lectures ou des écritures.

- Un bon sharding doit répartir les données, répartir la charge et permettre de localiser efficacement les données recherchées.

# Le sharding avec MongoDB



# Le sharding avec REDIS

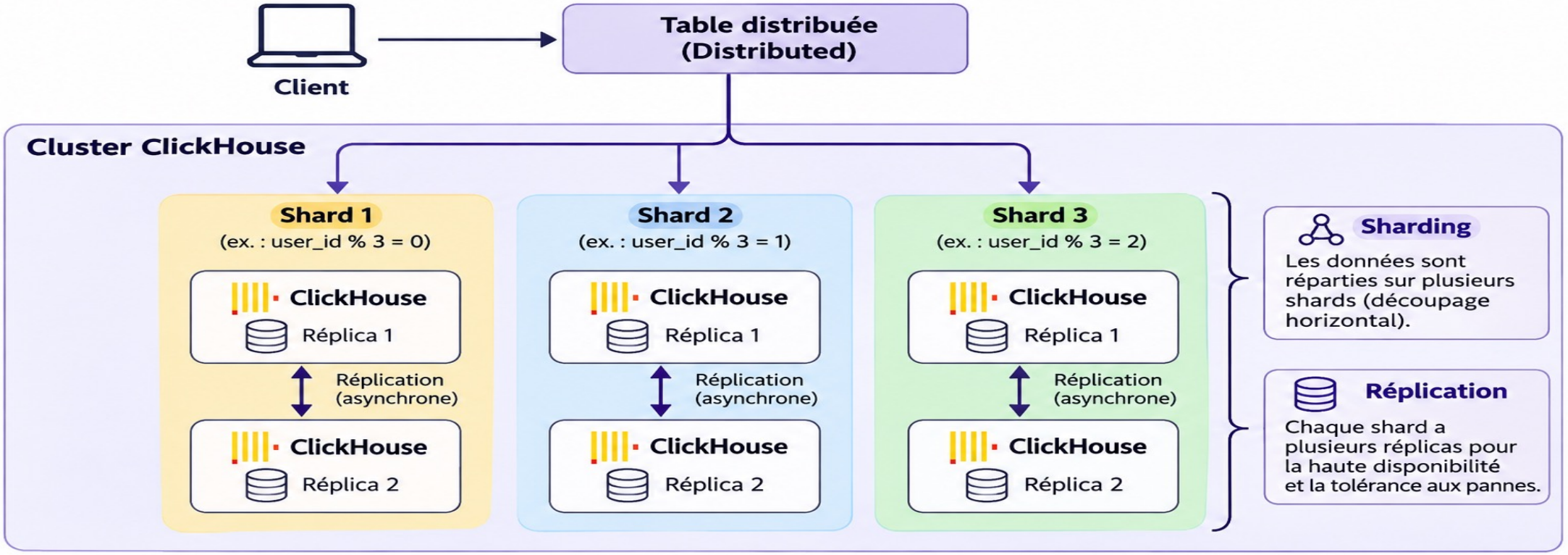


# Le sharding avec Clickhouse

ClickHouse

## Sharding et Réplication dans ClickHouse

Découpage horizontal des données (sharding) et haute disponibilité (réplication)




- **Scalabilité :** plus de données et plus de débit grâce au sharding.
- **Haute disponibilité :** continuité de service grâce à la réplication.
- **Tolérance aux pannes :** les réplicas prennent le relais en cas de défaillance.

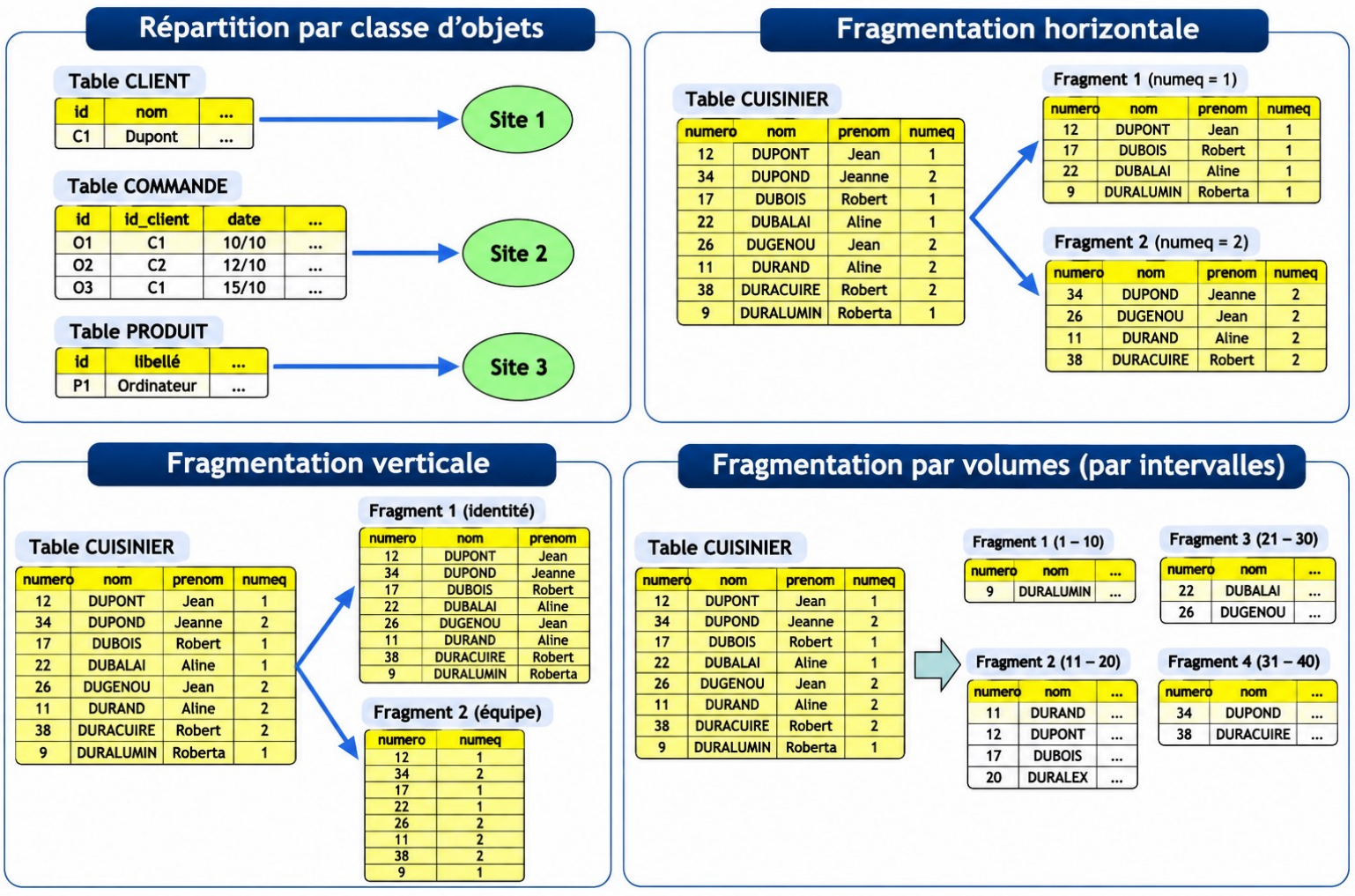
# TP 02 Sharding



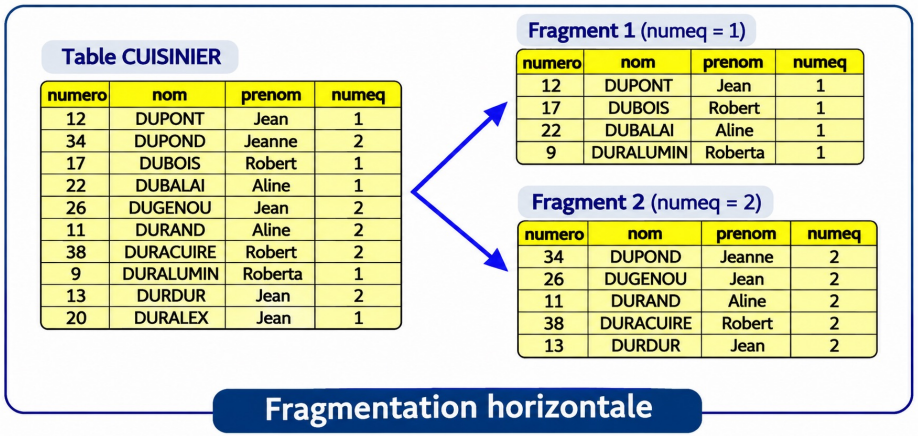
# La fragmentation



# La fragmentation



# La fragmentation horizontale




La **fragmentation horizontale** consiste à diviser une table en plusieurs sous-ensembles de lignes, appelés *fragments/partitions/chunks*.

Chaque fragment :

- Conserve les mêmes colonnes que la table d'origine.
- Contient uniquement les lignes satisfaisant certaines conditions.
- Peut être stocké sur un site ou un nœud différent.

**Répartir les données en fonction des besoins d'accès des utilisateurs**, afin de limiter les transferts de données et d'améliorer les performances des requêtes.

# Analyse des requêtes utilisateurs

**Identifier les requêtes les plus fréquentes**

Avant de fragmenter une table, il est nécessaire d'analyser les requêtes exécutées par les utilisateurs.

Prenons l'exemple de la table :

- **CUISINIER(numero, nom, prenom, numeq)**
- Les trois requêtes les plus fréquentes sont :

**Table CUISINIER**

| numero | nom | prenom | numeq |
|---|---|---|---|
| 12 | DUPONT | Jean | 1 |
| 34 | DUPOND | Jeanne | 2 |
| 17 | DUBOIS | Robert | 1 |
| 22 | DUBALAI | Aline | 1 |
| 26 | DUGENOU | Jean | 2 |
| 11 | DURAND | Aline | 2 |
| 38 | DURACUIRE | Robert | 2 |
| 9 | DURALUMIN | Roberta | 1 |
| 13 | DURDUR | Jean | 2 |
| 20 | DURALEX | Jean | 1 |

PK

- R1 : SELECT numero, numeq FROM CUISINIER WHERE prenom = 'Jean' AND nom LIKE '%R%';
- R2 : SELECT * FROM CUISINIER WHERE numeq = '1';
- R3 : SELECT numero, nom FROM CUISINIER WHERE numeq = '2' AND prenom = 'Jean';

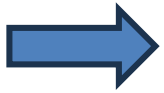
- **À retenir :** les clauses WHERE permettent d'identifier les critères susceptibles de servir à la fragmentation horizontale.

# Identification des conditions de sélection

- R1 : SELECT numero, numeq FROM CUISINIER WHERE prenom = 'Jean' AND nom LIKE '%R%';
- R2 : SELECT * FROM CUISINIER WHERE numeq = '1';
- R3 : SELECT numero, nom FROM CUISINIER WHERE numeq = '2' AND prenom = 'Jean';

- À partir des requêtes précédentes, nous identifions les conditions de sélection

| Requête | Condition de sélection |
|---|---|
| R1 | prenom = 'Jean' AND nom LIKE '%R%' |
| R2 | numeq = 1 |
| R3 | numeq = 2 AND prenom = 'Jean' |



| Prédicat | Condition |
|---|---|
| A | prenom = 'Jean' |
| B | nom LIKE '%R%' |
| C | numeq = 1 |
| D | numeq = 2 |

Chaque prédicat est élémentaire, il représente une conditions simple. Plusieurs prédicats peuvent être combinés pour former une condition de sélection.
Les conditions des requêtes deviennent :
C1=A ^ B
C2=C
C3=D ^ A

# Formalisation des conditions logiques

- Pour construire les fragments, nous utilisons les opérateurs de la logique booléenne.

| Opérateur | Symbole | Signification |
|---|---|---|
| ET | ^ | Les deux conditions doivent être vraies |
| OU | v | Au moins une condition doit être vraie |
| NON | - | Inverse la valeur logique d'une condition |

Exemple
- A : prenom='Jean'
- D : numeq=2
- L'expression : A ^ D

sélectionne les cuisiniers prénommés Jean appartenant à l'équipe 2.

- -A ^D : prenom <> 'Jean' et numeq=2

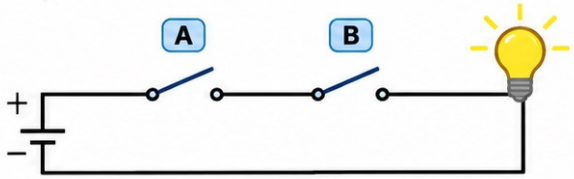
sélectionne les cuisiniers de l'équipe 2 dont le prénom n'est pas Jean.

# Simplification des expressions logiques

- Rappel sur les opérateurs logiques

## L'opérateur ET (∧) – AND

- Deux interrupteurs **en série**.
- L'ampoule s'allume **seulement si les deux** interrupteurs sont fermés.



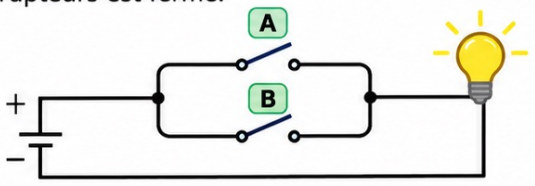

| A | B | A ∧ B (ampoule) |
|---|---|---|
| 0 | 0 | Éteinte |
| 0 | 1 | Éteinte |
| 1 | 0 | Éteinte |
| 1 | 1 | Allumée |

**Exemple SQL :**

```
SELECT *
FROM CUISINIER
WHERE prenom = 'Jean'
  AND numeq = 2;
```

## L'opérateur OU (∨) – OR

- Deux interrupteurs en **parallèle**.
- L'ampoule s'allume **si au moins un** des deux interrupteurs est fermé.




| A | B | A ∨ B (ampoule) |
|---|---|---|
| 0 | 0 | Éteinte |
| 0 | 1 | Allumée |
| 1 | 0 | Allumée |
| 1 | 1 | Allumée |

**Exemple SQL :**

```
SELECT *
FROM CUISINIER
WHERE prenom = 'Jean'
  OR numeq = 2;
```

chaque interrupteur représente une condition, et l'ampoule allumée signifie que la condition logique est vraie

# Simplification des expressions logiques

- Ø signifie qu'aucun élément ne satisfait la condition, donc la condition est fausse

**Table de vérité (OU)**

| A | B | A ∨ B |
|---|---|---|
| Ø | Ø | Ø |
| Ø | 1 | 1 |
| 1 | Ø | 1 |
| 1 | 1 | 1 |

**Table de vérité (ET)**

| A | B | A ∧ B |
|---|---|---|
| Ø | Ø | Ø |
| Ø | 1 | Ø |
| 1 | Ø | Ø |
| 1 | 1 | 1 |

- Les lois de De Morgan permettent de transformer et de simplifier les expressions logiques.
  - -(X∧Y) = (-X∨-Y)
  - -(X∨Y) =(-X∧-Y)

Développer l'expression suivante : A ^ -( B ∧ -A)

= A ^ (-B v A)

= (A ^ -B ) v (A ^ -A)

= A ^ -B

# Fragementation horizontale

Rappel des conditions par requête :

- C1=A ^ B
- C2=C
- C3=D ^ A

A partir des conditions Ci, on peut construire l'ensemble des conjonctions CCj de conditions :

- CC = { ∧Ci* pour i = 1,n et Ci* = Ci ou ¬Ci}
- CC = {C1∧C2∧C3, ¬C1∧C2∧C3, C1∧¬C2∧C3, C1∧C2∧¬C3, ¬C1∧¬C2∧C3, C1∧¬C2∧¬C3, ¬C1∧C2∧¬C3, ¬C1∧¬C2∧¬C3}

On peut faire le constat que C∧D = ∅, ¬C∧D = D, C∧¬D = C

- C∧D => numeq = '1' and numeq = '2' => ∅ => C∧D = ∅
- ¬C∧D => numeq <> '1' and numeq = '2' => numeq = '2' => ¬C∧D = D
- C∧¬D => numeq = '1' and numeq <> '2' => numeq = '1' => C∧¬D = C

On peut donc en conclure que :

- C1∧C2∧C3 = A∧B∧C∧D∧ A = A∧B∧∅∧ A = ∅
- ¬C1∧C2∧C3 = ¬(A∧B)∧C∧D∧A = ¬(A∧B)∧∅∧A = ∅
- C1∧C2∧¬C3 = A∧B∧C∧¬(D∧ A) = A∧B∧C∧(¬D∨¬A) = (A∧B∧C∧¬D)∨(A∧B∧C∧¬A) = (A∧B∧C)∨∅ = A∧B∧C

# Fragementation horizontale

On obtient 5 conjonctions de conditions :

CC1 = C1∧¬C2∧C3 = A∧B∧D
CC2 = C1∧C2∧¬C3 = A∧B∧C
CC3 = ¬C1∧¬C2∧C3 = A∧¬B∧D
CC4 = ¬C1∧C2∧¬C3 = (¬A∨¬B)∧C
CC5 = ¬C1∧¬C2∧¬C3 = ¬A∧D

Rappel
A : prenom = 'Jean'
B : nom LIKE '%R%
C : numeq = '1'
D : numeq = '2'

- R1 : SELECT numero, numeq FROM CUISINIER WHERE prenom = 'Jean' AND nom LIKE '%R%';
- R2 : SELECT * FROM CUISINIER WHERE numeq = '1';
- R3 : SELECT numero, nom FROM CUISINIER WHERE numeq = '2' AND prenom = 'Jean';

**Table CUISINIER**

| numero | nom | prenom | numeq |
|---|---|---|---|
| 12 | DUPONT | Jean | 1 |
| 34 | DUPOND | Jeanne | 2 |
| 17 | DUBOIS | Robert | 1 |
| 22 | DUBALAI | Aline | 1 |
| 26 | DUGENOU | Jean | 2 |
| 11 | DURAND | Aline | 2 |
| 38 | DURACUIRE | Robert | 2 |
| 9 | DURALUMIN | Roberta | 1 |
| 13 | DURDUR | Jean | 2 |
| 20 | DURALEX | Jean | 1 |

PK

CC1 — Fragment 1

| numero | nom | prenom | numeq |
|---|---|---|---|
| 13 | DURDUR | Jean | 2 |

→ R1 R3

CC2 — Fragment 2

| numero | nom | prenom | numeq |
|---|---|---|---|
| 20 | DURALEX | Jean | 1 |

→ R1 R2

CC3 — Fragment 3

| numero | nom | prenom | numeq |
|---|---|---|---|
| 26 | DUGENOU | Jean | 2 |

→ R3

CC4 — Fragment 4

| numero | nom | prenom | numeq |
|---|---|---|---|
| 12 | DUPONT | Jean | 1 |
| 17 | DUBOIS | Robert | 1 |
| 22 | DUBALAI | Aline | 1 |
| 9 | DURALUMIN | Roberta | 1 |

→ R2

CC5 — Fragment 5

| numero | nom | prenom | numeq |
|---|---|---|---|
| 34 | DUPOND | Jeanne | 2 |
| 11 | DURAND | Aline | 2 |
| 38 | DURACUIRE | Robert | 2 |

→ rien

# Les règles à vérifier

La fragmentation doit vérifier les propriétés suivantes :

- **Complétude** : toutes les lignes de la table d'origine doivent être représentées.
- **Disjonction** : chaque ligne doit appartenir à un seul fragment dans une fragmentation horizontale primaire disjointe.
- **Reconstruction** : l'union des fragments doit permettre de retrouver la table d'origine.

# TD1

On considère une base de données relationnelle réduite à une seule table relative à des représentants de commerce se partageant le territoire français en deux territoires « Nord » et « Sud ». Un exemple de cette base de données est donné ci-dessous

**REPRESENTANT**

| numRep | nomRep | salRep | tauxRep | territoire |
|---|---|---|---|---|
| 10 | DUPUIS | 1200 | 5 | Nord |
| 11 | DUBALAI | 1300 | 6 | Nord |
| 12 | DUCHEMIN | 1250 | 4 | Sud |
| 13 | DUMESNIL | 1400 | 5 | Sud |
| 14 | DUBOIS | 1300 | 4 | Sud |
| 15 | DURACUIRE | 1400 | 5 | Nord |
| 16 | DUPONT | 1500 | 6 | Nord |

SalRep et tauxRep représentent le salaire mensuel de base et le taux de commission (en pourcentage) respectivement.

On constate que les requêtes les plus souvent formulées sont :

- R1 : SELECT numRep, nomRep FROM REPRESENTANT WHERE territoire = 'Nord' ;
- R2 : SELECT * FROM REPRESENTANT WHERE nomRep LIKE '%S%' ;
- R3 : SELECT nomRep, salRep, tauxRep FROM REPRESENTANT WHERE territoire= 'Sud ' AND nomRep LIKE '%S% ‘ ;

On désire en premier lieu procéder à une fragmentation horizontale. Pour cela on calculera toutes les 8 conjonctions de conditions des clauses WHERE :

- CC = {C1*∧ C2*∧C3* avec Ci* = Ci ou ¬Ci}

On posera

- A : territoire = « Nord »
- B : territoire = « Sud »
- C : nomRep LIKE « %S% »

Déterminer la fragmentation horizontale optimale ?

# TD2

| id | destination | poids | prioritaire |
|---|---|---|---|
| 1 | Laon | 10 | non |
| 2 | Soissons | 5 | oui |
| 3 | Lille | 8 | non |
| 4 | Bordeaux | 12 | non |
| 5 | Lyon | 4 | oui |
| 6 | Nantes | 48 | oui |
| 7 | Amiens | 20 | non |
| 8 | Reims | 15 | oui |
| 9 | Paris | 2 | oui |

On considère la table colis (ci-dessus) d'une base de données relationnelle listant les envois de colis ou lettres.
Elle contient la destnation d'envoi, le poids et si elle est prioritaire ou non.

On souhaite répartir ceete table en se basant sur les trois requêtes les plus utilisées :

- R1 : SELECT id, poids FROM colis WHERE prioritaire = 'oui'
- R2 : SELECT * FROM colis WHERE poids > 10
- R3 : SELECT priotaire, desnaon FROM colis WHERE prioritaire = 'non' AND poids <= 10

En posant :

- A : prioritaire = 'oui'
- B : poids > 10

Exprimer en fonction de A, B et leurs contraires ¬A, ¬B, les condions C1, C2, C3 des trois requêtes précédentes.
Calculer les 8 conjoncons CCj des condions C1, C2 et C3 .
Déduire du résultat précédent et des données de la table colis les fragments horizontaux.